# PS1 — AI-Driven Energy Supply Chain Resilience

## Full Build Guide — Timed to Submission Window (18 Aug, 1:00 PM → 23 Aug, 4:00 PM IST)

Total window: ~5 days, 3 hours (123 hrs wall-clock).

**Deliverables required at submission:**

1. Prototype link (hosted preferred) or clear local-run README
2. GitHub repo, well-documented README
3. Demo video, MANDATORY, ≤10 minutes
4. Judged on: innovation, technical implementation, feasibility, scalability, code quality, documentation, presentation

Documentation and presentation are explicitly graded — budget real time for README and video, not just leftover hours on Day 5.

---

## 1. Scope Decision — build 3 of 5, one connected pipeline

Don't attempt all 5 illustrative directions. Build these 3 as one connected system:

1. **Geopolitical Risk Intelligence Agent** — live disruption-probability score per corridor/supplier

2. **Disruption Scenario Modeller** — pick a scenario, see cascading impact
3. **Adaptive Procurement Orchestrator** — ranked alternate routes/ suppliers with reasoning

Skip (mention as "v2" in the pitch): Strategic Reserve Optimisation Agent, full geospatial Digital Twin simulation engine. You still get the *visual* digital-twin feel for free from the map UI in component 1 — that covers the "innovation/presentation" impression without the extra engineering.

---

## 2. Architecture

```
┌──────────────────────────────────────────────────────────────
│ FRONTEND (Next.js + Tailwind)
│  - Map (Mapbox GL / deck.gl): corridors + suppliers,
│    color-coded by live risk score
│  - Scenario Simulator panel
│  - Recommendation panel (ranked routes, reasoning)
│  - Risk trend charts (Recharts)
└───────────────▲──────────────────────────────────────────────
                │ REST API
┌───────────────┴──────────────────────────────────────────────
│ BACKEND (FastAPI or Node/Express)
│  - Scheduled job: pull news + sanctions + energy data
│  - /risk-score: LLM scores corridors from cached news
│  - /simulate: deterministic impact model + LLM narrativ
│  - /recommend: ranks alt. suppliers/routes + reasoning
└───────────────▲──────────────────────────────────────────────
                │
┌───────────────┴──────────────────────────────────────────────
│ DATA LAYER
│  - Postgres/Supabase: corridors, suppliers, risk histor
│  - Claude API: risk scoring, scenario narrative, reason
└──────────────────────────────────────────────────────────────
```

---

## 3. Live Data Sources

| Source | What you get | Access |
|---|---|---|
| **GDELT Project** | Global news events, geopolitical tone, real-time | Free, no key |
| **NewsAPI.org / The Guardian API** | Filtered headlines (Hormuz, Iran, Red Sea, sanctions) | Free tier, needs key |
| **EIA API** | Oil prices, trade flows, refining capacity | Free, needs key |
| **OFAC SDN List** | Current sanctioned entities/vessels | Free CSV |
| **PPAC India** | India crude import stats, refinery data | Free public reports |
| **MarineTraffic/ VesselFinder** | Shipping lane congestion | Freemium — polish only, not a core dependency |

**Practical tip:** pull on a schedule (every 10–15 min) into your DB, serve the dashboard from cached scores. Never make the demo depend on a live external API call succeeding in the moment.

---

## 4. LLM's Job (Claude API)

1. **Risk scoring** — batch of recent headlines per corridor → 0–100 disruption probability + justification + confidence, structured JSON.
2. **Scenario narrative** — given a chosen event (e.g. "Hormuz closure") + current supply data → cascading-impact narrative (refining loss %, price delta, reserve-days impact). Pair with a

deterministic calculation — don't let the LLM invent the numbers alone.

3. **Recommendation reasoning** — given routes ranked by your own scoring formula (distance, cost proxy, current risk), LLM writes the "why" in procurement-officer language.

Keep the LLM as explainer/synthesizer, not sole number source — this is your strongest answer when judges ask "how do you know that number is real."

---

## 5. Day-by-Day Plan (5 days, 3 hrs)

**Day 1 (Aug 18, afternoon–evening)**

- Finalize scope (the 3 components above), pick 3–5 corridors/suppliers to model well (Hormuz, Red Sea, a couple of alternate routes) rather than the whole global network
- Scaffold Next.js + backend, deploy a skeleton early (Vercel/Render) so hosting isn't a Day 5 problem
- Get API keys sorted (NewsAPI, EIA), start pulling raw data

**Day 2 (Aug 19)**

- Build data ingestion pipeline: news + sanctions + energy data → DB, on a schedule
- Build /risk-score endpoint with the LLM scoring prompt, store history
- Rough map UI showing static corridors (even before live scores are wired in)

**Day 3 (Aug 20)**

- Wire live risk scores into the map (color-coded corridors)
- Build risk detail panel (click a corridor → see the headlines driving the score)
- Add risk trend chart

**Day 4 (Aug 21)**

- Build Scenario Simulator: 2–3 pre-defined scenarios (Hormuz closure, Red Sea suspension), deterministic impact model + LLM narrative
- Build Procurement Orchestrator: ranking formula + LLM reasoning panel
- Deploy final hosted version, test the live link end-to-end on a fresh browser

**Day 5 (Aug 22 → Aug 23 4PM)**

- Bug fixes, edge cases (API failure fallback — seed a cached snapshot so the demo never breaks on a dead API call)
- **Write the README properly** (see §7 — graded criterion)
- **Record the demo video** (see §6) with buffer time
- Final GitHub cleanup: comments, dead code removed, `git clone` → run instructions verified on a clean machine
- Submit early — don't risk the "late submissions not accepted" rule against last-minute server load

---

## 6. Demo Video Plan (≤10 min — aim for 5–7)

1. **0:00–0:45** — Problem framing: 88% import dependence, 40–45% via Hormuz, 9.5 days of reserves — use the brief's own numbers
2. **0:45–1:30** — Architecture in 3 sentences
3. **1:30–5:30** — Live walkthrough: map with real risk scores → click a red corridor → see driving headlines → run "Simulate Hormuz closure" → watch cascading impact populate → see top-3 recommended alternate routes with reasoning
4. **5:30–6:30** — Show the fallback/failure-handling case (proves robustness)
5. **6:30–7:30** — Scalability: more corridors, Reserve Optimizer, full digital twin as v2
6. **7:30–end** — Close on impact: compressing a days-long decision window into hours

Use real screen capture of the working app, not slides describing intended behavior — "technical implementation" and "feasibility" are scored on what's demonstrated.

---

## 7. README Checklist (explicitly graded)

- One-line problem + one-line solution at top
- Architecture diagram (simple is fine)
- Setup instructions verified on a clean clone
- `.env.example` listing required API keys
- Known limitations (which corridors/scenarios are supported — be upfront)
- Link to hosted prototype + link to demo video

---

## 8. Risk Mitigations

- **External API fails/rate-limits during demo** → always have a cached/seeded snapshot as fallback
- **LLM latency** → precompute risk scores on schedule; only the scenario narrative is a live call, with loading state + timeout fallback
- **Scope creep** → resist adding Reserve Optimizer/full digital twin unless the core 3 pieces are solid by ~Day 4
- **Numbers look made up** → always show the deterministic calc alongside the LLM narrative, and cite the actual headlines behind each risk score

---

## 9. Pitch Framing

- **Innovation**: real-time geopolitical signal → quantified corridor risk → actionable procurement recommendation, as one connected pipeline

- **Technical implementation**: hybrid deterministic-model + LLM-reasoning approach, not "we asked an LLM for a number"
- **Feasibility**: built on real, live data sources (GDELT, EIA, OFAC), not synthetic
- **Impact**: compresses a procurement decision window from days to hours against India's 9.5-day reserve buffer
- **Scalability**: clear v2 path — Reserve Optimization Agent, full geospatial digital twin, more corridors/suppliers